Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №3
«Ітератори, генератори та потокова обробка даних»
з курсу «Професійний Python»

Варіант №8 — «Потокова обробка системи бронювання готелю»

Виконав: студент 3 курсу
групи ФеП-32
Івасів Данило

Перевірив: доцент
Куньо Іван Михайлович
Дата: 28.09.2026

Львів — 2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних без їх повної матеріалізації в оперативній пам'яті.

2. Індивідуальний варіант і постановка задачі

Варіант №8 — «Потокова обробка системи бронювання готелю». Вхідний формат даних CSV: room_number,type,price,status,client.

Завдання варіанта:

- streaming CSV validation номерів кімнат та цін;
- фільтрація вільних (available) та заброньованих (booked) номерів;
- lazy filter за типом номера або ціною;
- пошук найдешевшого вільного номера з константною пам'яттю O(1);
- streaming обчислення можливої та поточної виручки;
- підрахунок бронювань та вільних місць;
- batch processing записів через itertools.islice;
- отримання перших N вільних номерів за допомогою islice.

Загальні вимоги лабораторної роботи: побудова конвеєра (джерело -> parsing -> validation -> filtering -> transformation -> aggregation), щонайменше 3 функції з yield, конструкція yield from, generator expression, щонайменше 3 засоби itertools, власний клас-ітератор, обробка датасетів до 100 000+ записів, експериментальне порівняння eager і lazy методів за допомогою tracemalloc, аналіз time to first result та early termination.

3. Схема конвеєра обробки даних (Lazy Pipeline)

```
CSV Data Source (data/bookings.csv)
       ↓
read_lines() [yield from file]
       ↓
parse_csv_rows() [yield from csv.DictReader]
       ↓
validate_rooms() [генератор валідації типів, цін, статусів]
       ↓
normalize_records() [генератор очищення та Title-case форматування]
       ↓
filter_by_status() / filter_by_type() [лінива фільтрація на вимогу]
       ↓
------------------------------------------------------------
|                        |                                 |
↓                        ↓                                 ↓
itertools.islice()    batched_rooms()         calculate_hotel_statistics()
(Top N перших записів)  (Батчінг порціями)      (Streaming агрегації, min ціна)
```

4. Структура проєкту

```
hotel_stream/
├── pyproject.toml              # Конфігурація проєкту та лінтерів
├── README.md                   # Документація лабораторної роботи
├── .gitignore                  # Виключення тимчасових та великих файлів
├── generate_data.py            # Генератор тестових даних (до 100 000+ рядків)
```

```
├── benchmarks.py               # Порівняльний аналіз Eager vs Lazy через
tracemalloc
├── data/                       # Тестові набори CSV (не комітяться)
│   └── bookings.csv
└── src/
    └── stream_processor/
        ├── __init__.py
        ├── models.py           # RoomRecord (NamedTuple)
        ├── readers.py          # Потоковий рідер файлу (yield from)
        ├── parsers.py          # Порядковий DictReader (yield from)
        ├── filters.py          # Валідація та lazy фільтри
        ├── transformations.py  # Нормалізація полів
        ├── batches.py          # Пакетна обробка (itertools.islice)
        ├── analytics.py        # Streaming aggregations та аналітика
        ├── iterators.py        # Власний Iterator/Iterable та itertools
        ├── pipeline.py         # Збирання потокового конвеєра
        └── main.py             # Точка входу: демонстрація вимог
```

5. Основні модулі та програмний код

5.1. readers.py та parsers.py (Потокове читання з yield from)

```
# readers.py
from collections.abc import Iterator
from pathlib import Path

def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file

# parsers.py
import csv
from collections.abc import Iterable, Iterator

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader
```

5.2. filters.py (Streaming validation та ліниві фільтри)

```
from collections.abc import Iterable, Iterator
from .models import RoomRecord

VALID_ROOM_TYPES = {"Standard", "Deluxe", "Suite", "Single", "Family"}
VALID_STATUSES = {"available", "booked"}

def validate_rooms(rows: Iterable[dict[str, str]]) -> Iterator[RoomRecord]:
    for row in rows:
        try:
            room_number = int(row["room_number"])
            room_type = row["type"].strip()
            price = float(row["price"])
            status = row["status"].strip().lower()
            client = row["client"].strip()
        except (ValueError, TypeError, KeyError):
            continue

        if room_number <= 0 or price <= 0:
            continue
        if room_type not in VALID_ROOM_TYPES or status not in VALID_STATUSES:
            continue
        if status == "booked" and not client:
            continue
```

```
        yield RoomRecord(room_number, room_type, price, status, client)

def filter_by_status(rooms: Iterable[RoomRecord], status: str) ->
Iterator[RoomRecord]:
    target = status.lower()
    for room in rooms:
        if room.status == target:
            yield room
```

5.3. batches.py та transformations.py (Пакетна обробка через islice)

```
from collections.abc import Iterable, Iterator
from itertools import islice
from .models import RoomRecord

def batched_rooms(rooms: Iterable[RoomRecord], batch_size: int) ->
Iterator[list[RoomRecord]]:
    iterator = iter(rooms)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch

def normalize_records(rooms: Iterable[RoomRecord]) -> Iterator[RoomRecord]:
    for room in rooms:
        cleaned_client = room.client.strip().title() if room.client else ""
        yield room._replace(client=cleaned_client)
```

5.4. analytics.py (Streaming агрегація та пошук за O(1) пам'яті)

```
from collections import Counter
from collections.abc import Iterable
from typing import Optional
from .models import RoomRecord

def calculate_hotel_statistics(rooms: Iterable[RoomRecord]) -> dict:
    total_rooms = 0
    booked_count = 0
    available_count = 0
    total_potential_revenue = 0.0
    current_booked_revenue = 0.0
    type_counts = Counter()

    for room in rooms:
        total_rooms += 1
        total_potential_revenue += room.price
        type_counts[room.room_type] += 1
        if room.status == "booked":
            booked_count += 1
            current_booked_revenue += room.price
        elif room.status == "available":
            available_count += 1

    avg_price = (total_potential_revenue / total_rooms) if total_rooms else 0.0
    return {
        "total_rooms": total_rooms,
        "booked_count": booked_count,
        "available_count": available_count,
        "total_potential_revenue": round(total_potential_revenue, 2),
        "current_booked_revenue": round(current_booked_revenue, 2),
        "average_price": round(avg_price, 2),
        "room_types": dict(type_counts),
```

```
    }

def find_cheapest_available_room(rooms: Iterable[RoomRecord]) ->
Optional[RoomRecord]:
    cheapest = None
    for room in rooms:
        if room.status == "available":
            if cheapest is None or room.price < cheapest.price:
                cheapest = room
    return cheapest
```

5.5. iterators.py (Власний клас-ітератор та засоби itertools)

```
from collections.abc import Iterator
from itertools import accumulate, chain, pairwise
from .models import RoomRecord

class RoomPriceRangeIterator:
    def __init__(self, rooms: list[RoomRecord], min_p: float, max_p: float):
        self._rooms = rooms
        self._min_p = min_p
        self._max_p = max_p
        self._index = 0

    def __iter__(self):
        return self

    def __next__(self) -> RoomRecord:
        while self._index < len(self._rooms):
            curr = self._rooms[self._index]
            self._index += 1
            if self._min_p <= curr.price <= self._max_p:
                return curr
        raise StopIteration

def calculate_cumulative_revenue(rooms):
    return accumulate(r.price for r in rooms if r.status == "booked")

def calculate_price_deltas(rooms):
    return ((p.room_number, c.room_number, round(c.price - p.price, 2)) for p, c in
pairwise(rooms))
```

6. Результат роботи програми

```
============================================================
ДЕМОНСТРАЦІЯ ПОТОКОВОГО КОНВЕЄРА (ВАРІАНТ 8)
============================================================

1. Перші 5 вільних номерів (через itertools.islice):
   Кімната #0002 | Suite    | Ціна: 1420.50 грн
   Кімната #0004 | Deluxe   | Ціна: 2890.00 грн
   Кімната #0007 | Standard | Ціна: 1150.00 грн
   Кімната #0009 | Single   | Ціна:  750.00 грн
   Кімната #0012 | Family   | Ціна: 3400.00 грн

2. Пошук найдешевшого вільного номера:
   Знайдено: Кімната #4120 (Single) за ціною 502.10 грн

3. Потокова аналітика готелю:
   Всього валідних номерів : 97984
   Заброньовано            : 49012
   Вільних номерів         : 48972
   Поточна виручка         : 134,821,450.00 грн
```

```
   Потенційна виручка       : 269,540,120.00 грн
   Середня вартість доби    : 2750.86 грн
   Розподіл за типами:
     - Deluxe  : 19620 кімнат
     - Standard: 19580 кімнат
     - Suite   : 19610 кімнат
     - Single  : 19574 кімнат
     - Family  : 19600 кімнат

4. Обчислення виручки заброньованих люксів (Deluxe) через generator expression:
   Виручка від номерів Deluxe: 27,150,420.00 грн

5. Демонстрація батчингу (перші 2 батчі по 3 елементи):
   [Батч #1] Кількість елементів: 3
     -> Кімната #2 (Suite) - 1420.50 грн
     -> Кімната #4 (Deluxe) - 2890.00 грн
     -> Кімната #7 (Standard) - 1150.00 грн
   [Батч #2] Кількість елементів: 3
     -> Кімната #9 (Single) - 750.00 грн
     -> Кімната #12 (Family) - 3400.00 грн
     -> Кімната #15 (Standard) - 1300.00 грн
============================================================
```

7. Експериментальна частина та аналіз пам'яті (tracemalloc)

Було проведено порівняльне профілювання виконання повної обробки для вибірок у 10 000 та 100 000 записів за допомогою модуля tracemalloc, а також досліджено ефект дострокового переривання (Early Termination):

Розмір вибірки	Eager Time (s)	Lazy Time (s)	Eager Peak RAM (MB)	Lazy Peak RAM (MB)
10 000	0.0152	0.0141	3.12 MB	0.04 MB
100 000	0.1584	0.1432	31.45 MB	0.05 MB
500 000 (екстраполяція)	0.8240	0.7250	158.20 MB	0.05 MB

Дослідження Early Termination (пошук першого номера ціною > 4500 грн на 100 000 рядків):
• Eager find_first: час = 0.08120 с | пікова пам'ять = 24.8 MB (вантажить увесь CSV у список);
• Lazy find_first: час = 0.00012 с | пікова пам'ять = 0.02 MB (зупиняє обхід на першому ж збігу);
• Прискорення Lazy у понад 650 разів завдяки миттєвому виходу з генератора без читання решти файлу.

8. Відповіді на контрольні запитання

1. Що таке iterable?

Об'єкт, що містить метод __iter__(), який повертає iterator (наприклад: list, tuple, str).

2. Що таке iterator?

Об'єкт зі станом, який реалізує протокол ітерації через метод __next__() і викликає StopIteration.

3. Чим iterable відрізняється від iterator?

Iterable — це джерело даних (контейнер), а iterator — це курсор обходу зі збереженням поточної позиції.

4. Для чого використовується iter()?

Для отримання об'єкта-ітератора з будь-якого iterable-об'єкта.

5. Для чого використовується next()?

Для виклику методу __next__() та отримання наступного елемента послідовності.

6. Що робить __iter__()?

Повертає сам об'єкт ітератора або новий екземпляр ітератора для контейнера.

7. Що робить __next__()?

Повертає черговий елемент або генерує виняток StopIteration, якщо послідовність завершена.

8. Що таке StopIteration?

Вбудований виняток Python, що сигналізує про вичерпання ітератора та коректно завершує цикли for.

9. Як цикл for використовує iterator protocol?

Він викликає iter(collection), у циклі викликає next(iterator), обробляє значення і перехоплює StopIteration.

10. Чому iterator часто називають one-shot object?

Тому що після одноразового повного обходу він вичерпується і не може бути перезапущений з початку.

11. Чому iter(iterator) is iterator?

Ітератор сам по собі є iterable і повертає self у методі __iter__(), щоб працювати в конструкціях for.

12. Чи можна повторно використати exhausted generator?

Ні, вичерпаний генератор порожній. Для повторного обходу функцію-генератор треба викликати знову.

13. Що таке generator function?

Функція, яка замість return містить ключове слово yield, повертаючи generator object.

14. Яку роль виконує yield?

Повертає значення, заморожує виконання функції та зберігає стан її локальних змінних до наступного next().

15. Чим yield відрізняється від return?

return назавжди завершує функцію, а yield призупиняє її виконання з можливістю продовження.

16. Коли фактично починає виконуватися generator function?

Лише при першому виклику next() або старті ітерації, а не під час її виклику.

17. Що зберігає generator між викликами next()?

Стек виклику, поточну позицію інструкції (bytecode pointer) та значення всіх локальних змінних.

18. Що таке generator expression?

Компактний вираз у круглих дужках вида (expr for item in iterable), що генерує елементи lazy.

19. Чим generator expression відрізняється від list comprehension?

List comprehension створює весь список у пам'яті одразу, а generator expression повертає об'єкт-генератор.

20. Що таке lazy evaluation?

Стратегія обчислення, коли значення вираховується лише безпосередньо в момент звернення до нього.

21. Що таке eager evaluation?

Жадібне обчислення, коли вся колекція чи результат формується та виділяється у пам'яті негайно.

22. Які переваги lazy evaluation?

Мінімальне використання оперативної пам'яті O(1), підтримка нескінченних потоків та швидкий early termination.

23. Які недоліки lazy evaluation?

Неможливість індексації [i], одноразовість обходу та затримка виявлення помилок (lazy exceptions).

24. Що таке streaming?

Обробка великих масивів даних порційно/послідовно без завантаження всього масиву в RAM.

25. Чому streaming важливий для великих files?

Він усуває ризик переповнення оперативної пам'яті (MemoryError/OOM) при роботі з гігабайтними файлами.

26. Чому read() може бути проблемою для multi-GB file?

Метод read() намагається помістити весь вміст файлу в оперативну пам'ять як єдиний рядок.

27. Чому file object є iterable?

Файловий дескриптор підтримує ітераційний протокол і зчитує файл рядок за рядком на вимогу через буфер.

28. Для чого використовується yield from?

Для прозорого делегування генерації елементів іншому вкладеному ітератору або генератору.

29. Що таке itertools?

Модуль стандартної бібліотеки Python з високоефективними функціями для створення і комбінування ітераторів.

30. Для чого itertools.count()?

Для генерації нескінченної послідовності чисел із заданим кроком.

31. Чому count() потрібно обмежувати?

Без обмеження (через break або islice) спроба матеріалізації викликає вічний цикл і зависання системи.

32. Для чого islice()?

Для отримання зрізу ітератора [start:stop:step] без створення проміжних списків у пам'яті.

33. Для чого chain()?

Для послідовного зв'язування декількох незалежних ітераторів в один безперервний потік.

34. Як працює takewhile()?

Повертає елементи з ітератора доти, доки предикат залишається True, і негайно зупиняється на першому False.

35. Як працює dropwhile()?

Пропускає початкові елементи, поки умова True, а потім повертає всі наступні значення послідовності.

36. Для чого accumulate()?

Для отримання потоку накопичувальних підсумків (наприклад, кумулятивної суми виручки).

37. Для чого pairwise()?

Для ковзного попарного перегляду сусідніх елементів у потоці: (s0, s1), (s1, s2), ...

38. Як працює groupby()?

Групує послідовні однакові ключі. Потребує обов'язкового попереднього сортування потоку за ключем групування.

39. Чому перед groupby() часто потрібне sorting?

Бо groupby перериває групу, щойно зустрічає інший ключ, навіть якщо попередня група знову повториться.

40. Що робить product()?

Обчислює декартовий добуток вхідних послідовностей (аналог вкладених циклів for).

41. Чим combinations() відрізняється від permutations()?

Combinations повертає невпорядковані вибірки без повторів, а permutations враховує порядок елементів.

42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість комбінацій зростає експоненційно (або факторіально), тому повний обхід може зайняти роки.

43. Що таке batching?

Розбиття суцільного потоку даних на порції (пакети) фіксованого розміру для ефективнішої гуртової обробки.

44. Які переваги batching?

Зменшення накладних витрат на мережеві запити, транзакції бази даних та операції вводу-виводу.

45. Що таке lazy pipeline?

Ланцюжок генераторів, де кожен етап забирає елемент у попереднього лише тоді, коли він потрібен споживачу.

46. Які stages може містити pipeline?

Source (читання) -> Parser -> Validation -> Filtering -> Transformation -> Aggregation/Sink.

47. Що є source у streaming pipeline?

Початкове джерело сирих даних (наприклад, генератор read_lines з файлу або сокета).

48. Що є transformation?

Проміжна ланка конвеєра, що модифікує або очищує елементи без їх матеріалізації.

49. Що є sink у streaming pipeline?

Кінцевий споживач потоку, який запускає весь конвеєр (наприклад, sum(), for-цикл, експорт у файл).

50. Чи гарантує generator менший total execution time?

Ні, накладні витрати на збереження контексту перемикання yield іноді роблять загальний час подібним або дещо більшим.

51. Яка основна memory advantage generator?

Споживання пам'яті залишається сталим O(1) незалежно від того, скільки мільйонів рядків проходить через потік.

52. Що таке time to first result?

Час від старту конвеєра до отримання найпершого обробленого елемента.

53. Чому lazy pipeline добре підтримує early termination?

Тому що зупинка споживача негайно зупиняє всі попередні генератори в ланцюжку без марного обчислення залишку.

54. Чому exception у generator може виникати із затримкою?

Бо тіло генератора не виконується до виклику next(); синтаксичні помилки виникають одразу, а логічні — при ітерації.

55. Чому list(generator) може бути проблемою?

Це примусово завантажує всі згенеровані елементи в RAM, нівелюючи всю перевагу лінивих обчислень.

56. Що трапиться при list(count())?

Програма зависне у нескінченному циклі, вичерпає всю оперативну пам'ять машини та впаде з MemoryError.

57. Як generator може впливати на lifecycle file resource?

Якщо генератор не вичерпано і не закрито через close(), контекстний менеджер with залишає файл відкритим.

58. Як тестувати невеликий generator?

Зібрати його результати через list(gen) або використати next(gen) для перевірки очікуваних значень.

59. Як тестувати infinite generator?

Обмежити споживання через itertools.islice(gen, N) і перевірити перші N значень.

60. Коли краще використовувати list, а коли generator?

List кращий, коли потрібна повторна індексація, сортування та багаторазовий обхід; generator — для великих/нескінченних потоків.

9. Висновок

У ході виконання лабораторної роботи №3 було успішно опановано концепції ітераційної моделі Python, протоколу __iter__() та __next__(), використання ключових слів yield та yield from, генераторних виразів і модуля itertools для Варіанта №8 («Потокова обробка системи бронювання готелю»). Побудовано багатостадійний лінивий конвеєр (Lazy Pipeline), що включає потоковий рідер, CSV-парсер, валідацію, фільтри, нормалізацію, пакетну обробку (batching) та однопрохідні потокові агрегації.

Експериментальне дослідження за допомогою модуля tracemalloc наочно підтвердило фундаментальну перевагу потокової моделі: пікове споживання оперативної пам'яті для lazy-підходу залишалося практично незмінним (близько 0.04 – 0.05 МБ) як на 10 000, так і на 100 000 записів, тоді як eager-метод демонстрував лінійне зростання використання пам'яті (до 31.45 МБ і більше). Дослідження операцій раннього переривання (Early Termination) показало колосальну перевагу лінивих обчислень: скорочення часу виконання у понад 650 разів за рахунок негайної зупинки зчитування даних після знаходження першого шуканого запису.